

BANCO DE DADOS COM MySQL

Aula introdutória: CRUD, consultas e organização dos dados

Material do professor • XAMPP + phpMyAdmin + MySQL

1. O que é um banco de dados?

Um banco de dados é uma forma organizada de armazenar informações para que elas possam ser cadastradas, consultadas, alteradas e removidas com facilidade.

Nesta aula, vamos imaginar um sistema simples de escola. O sistema precisa guardar dados dos alunos, como nome, idade e cidade.

Onde vamos trabalhar?

- XAMPP: inicia os serviços locais, principalmente Apache e MySQL.
- phpMyAdmin: interface acessada pelo navegador para administrar o MySQL.
- SQL: linguagem usada para enviar comandos ao banco de dados.

CRUD: as quatro operações básicas

Operação	Comando principal	O que faz
Create	CREATE / INSERT	Cria estruturas e cadastra dados.
Read	SELECT	Consulta e lê os dados.
Update	UPDATE	Altera dados existentes.
Delete	DELETE	Remove registros.

2. Criando o banco e a tabela

CREATE DATABASE — criar um banco

Podemos criar um banco chamado escola:

```
CREATE DATABASE escola;
```

Depois, selecionamos esse banco para trabalhar:

```
USE escola;
```

CREATE TABLE — criar uma tabela

Agora criaremos a tabela alunos. Cada coluna representa uma característica do aluno.

```
CREATE TABLE alunos (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    idade INT,  
    cidade VARCHAR(100)  
);
```

O que significa cada parte?

- id: identificador único de cada aluno.
- INT: número inteiro.
- AUTO_INCREMENT: gera o próximo número automaticamente.
- PRIMARY KEY: define o campo que identifica cada registro.
- VARCHAR(100): texto de até 100 caracteres.
- NOT NULL: o campo precisa receber um valor.

3. INSERT — cadastrando dados

O INSERT adiciona novos registros à tabela. É o comando usado quando queremos cadastrar um novo aluno.

Inserindo um aluno

```
INSERT INTO alunos (nome, idade, cidade)
VALUES ('Carlos Silva', 22, 'São Paulo');
```

Inserindo outro aluno

```
INSERT INTO alunos (nome, idade, cidade)
VALUES ('Mariana Souza', 19, 'Guarulhos');
```

Inserindo vários registros de uma vez

```
INSERT INTO alunos (nome, idade, cidade) VALUES
('Ana Lima', 25, 'São Paulo'),
('Bruno Costa', 18, 'Osasco'),
('Fernanda Alves', 21, 'Santo André');
```

Como o campo id possui AUTO_INCREMENT, não precisamos informar o número do aluno a cada novo cadastro.

4. SELECT — consultando os dados

SELECT é o comando utilizado para consultar informações. Ele não altera os registros; apenas mostra os dados encontrados.

Mostrar tudo

```
SELECT * FROM alunos;
```

O asterisco (*) significa “todas as colunas”.

Mostrar somente algumas colunas

```
SELECT nome, cidade FROM alunos;
```

Filtrar com WHERE

```
SELECT * FROM alunos  
WHERE cidade = 'São Paulo';
```

```
SELECT * FROM alunos  
WHERE idade = 22;
```

Comparações úteis

```
SELECT * FROM alunos WHERE idade > 20;
```

```
SELECT * FROM alunos WHERE idade >= 18;
```

```
SELECT * FROM alunos WHERE idade < 21;
```

Buscar parte de um texto com LIKE

```
SELECT * FROM alunos  
WHERE nome LIKE 'A%';
```

'A%' significa: nomes que começam com a letra A. Também podemos usar '%Silva%' para procurar a palavra Silva em qualquer parte do texto.

5. ORDER BY — organizando o resultado

ORDER BY altera apenas a ordem em que a consulta é exibida. Ele não muda a posição real dos registros no banco.

Ordem alfabética crescente — A até Z

```
SELECT * FROM alunos  
ORDER BY nome ASC;
```

Ordem alfabética decrescente — Z até A

```
SELECT * FROM alunos  
ORDER BY nome DESC;
```

Ordenar pela idade — menor para maior

```
SELECT * FROM alunos  
ORDER BY idade ASC;
```

Ordenar pela idade — maior para menor

```
SELECT * FROM alunos  
ORDER BY idade DESC;
```

Filtrar e ordenar ao mesmo tempo

```
SELECT * FROM alunos  
WHERE cidade = 'São Paulo'  
ORDER BY idade DESC;
```

ASC = ascending (crescente). DESC = descending (decrescente). Se ASC for omitido, a ordem crescente é o comportamento padrão.

6. UPDATE — alterando informações

UPDATE modifica dados que já existem. Normalmente usamos WHERE para indicar exatamente qual registro deve ser alterado.

Alterar a cidade de um aluno

```
UPDATE alunos  
SET cidade = 'Osasco'  
WHERE id = 1;
```

Alterar mais de uma coluna

```
UPDATE alunos  
SET idade = 23, cidade = 'Campinas'  
WHERE id = 1;
```

Alterar usando o nome como filtro

```
UPDATE alunos  
SET idade = 20  
WHERE nome = 'Mariana Souza';
```

Sempre que possível, prefira usar o id para localizar um registro específico, pois nomes podem se repetir.

ATENÇÃO: UPDATE sem WHERE pode alterar todos os registros da tabela.

```
UPDATE alunos SET cidade = 'São Paulo';
```

O exemplo acima colocaria São Paulo na coluna cidade de todos os alunos.

7. DELETE — excluindo registros

DELETE remove uma ou mais linhas (registros) da tabela.

Excluir um aluno pelo id

```
DELETE FROM alunos  
WHERE id = 2;
```

Excluir por uma condição

```
DELETE FROM alunos  
WHERE idade < 18;
```

ATENÇÃO: DELETE sem WHERE remove todas as linhas da tabela.

```
DELETE FROM alunos;
```

Linha, coluna e tabela são coisas diferentes

Quero remover...	Comando	Exemplo
Uma linha/registro	DELETE	DELETE FROM alunos WHERE id = 3;
Uma coluna	ALTER TABLE ... DROP COLUMN	ALTER TABLE alunos DROP COLUMN cidade;
A tabela inteira	DROP TABLE	DROP TABLE alunos;

8. ALTER TABLE e DROP — alterando a estrutura

Enquanto UPDATE altera os dados dentro das linhas, ALTER TABLE modifica a estrutura da tabela.

Adicionar uma coluna

```
ALTER TABLE alunos  
ADD email VARCHAR(150);
```

Remover uma coluna

```
ALTER TABLE alunos  
DROP COLUMN email;
```

Excluir a tabela inteira

```
DROP TABLE alunos;
```

DROP TABLE remove a estrutura da tabela e todos os dados armazenados nela.

TRUNCATE TABLE — apagar todos os registros

```
TRUNCATE TABLE alunos;
```

TRUNCATE esvazia a tabela, mas mantém sua estrutura. Em uma aula introdutória, basta apresentar a diferença geral entre DELETE, TRUNCATE e DROP.

Comando	Remove dados?	Mantém a tabela?
DELETE FROM alunos;	Sim	Sim
TRUNCATE TABLE alunos;	Sim, todos	Sim
DROP TABLE alunos;	Sim	Não

9. Mais consultas úteis para iniciantes

DISTINCT — não repetir valores

```
SELECT DISTINCT cidade FROM alunos;
```

Se vários alunos forem da mesma cidade, a cidade aparecerá apenas uma vez.

COUNT — contar registros

```
SELECT COUNT(*) FROM alunos;
```

```
SELECT COUNT(*) FROM alunos  
WHERE cidade = 'São Paulo';
```

LIMIT — limitar a quantidade mostrada

```
SELECT * FROM alunos  
LIMIT 3;
```

AND e OR — combinar condições

```
SELECT * FROM alunos  
WHERE idade >= 18 AND cidade = 'São Paulo';
```

```
SELECT * FROM alunos  
WHERE cidade = 'Osasco' OR cidade = 'Guarulhos';
```

BETWEEN — intervalo

```
SELECT * FROM alunos  
WHERE idade BETWEEN 18 AND 22;
```

10. Roteiro de demonstração completa

Você pode executar esta sequência ao vivo no projetor, explicando o que muda após cada comando.

1. Criar a tabela

```
CREATE TABLE alunos (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  idade INT,  
  cidade VARCHAR(100)  
);
```

2. Cadastrar alunos

```
INSERT INTO alunos (nome, idade, cidade) VALUES  
( 'Carlos Silva', 22, 'São Paulo'),  
( 'Mariana Souza', 19, 'Guarulhos'),  
( 'Ana Lima', 25, 'São Paulo');
```

3. Conferir os dados

```
SELECT * FROM alunos;
```

4. Colocar em ordem alfabética

```
SELECT * FROM alunos ORDER BY nome ASC;
```

5. Mostrar do mais velho ao mais novo

```
SELECT * FROM alunos ORDER BY idade DESC;
```

6. Alterar um registro

```
UPDATE alunos SET cidade = 'Osasco' WHERE id = 1;
```

7. Conferir a alteração

```
SELECT * FROM alunos WHERE id = 1;
```

8. Excluir um registro

```
DELETE FROM alunos WHERE id = 2;
```

9. Conferir o resultado final

```
SELECT * FROM alunos;
```

11. Resumo para deixar projetado no final

Comando	Função
CREATE DATABASE	Cria um banco.
CREATE TABLE	Cria uma tabela.
INSERT INTO	Cadastra registros.
SELECT	Consulta dados.
WHERE	Filtra os registros.
ORDER BY ... ASC	Ordena de forma crescente.
ORDER BY ... DESC	Ordena de forma decrescente.
UPDATE ... SET	Altera registros.
DELETE FROM	Exclui linhas/registros.
ALTER TABLE ... ADD	Adiciona coluna.
ALTER TABLE ... DROP COLUMN	Remove coluna.
TRUNCATE TABLE	Apaga todos os registros e mantém a tabela.
DROP TABLE	Exclui a tabela inteira.

Regra de ouro: antes de executar UPDATE ou DELETE, confira o WHERE. Uma boa prática é executar primeiro um SELECT com o mesmo WHERE para verificar quais registros serão afetados.

```
SELECT * FROM alunos WHERE id = 3;
-- conferiu? Então:
DELETE FROM alunos WHERE id = 3;
```